Lab 4: VibeCoding

AI Chat History & Work Log

Project: Tug of War — `56_tug_of_war`

Repository assigned: https://github.com/SETAPESU26/56_tug_of_war

Student GitHub account mentioned in troubleshooting: DharshiniPES

Purpose

This document records the key VibeCoding prompts and troubleshooting guidance from the ChatGPT conversation about completing Lab 4. It is a prepared work-log transcript, not an automatically exported, verbatim copy of the entire ChatGPT interface history. Review it and add any missing Copilot exchanges before submission.

Lab deliverables identified from the handout

• Record a 10-second video of the original game before edits.

• Use AI prompts to fix the broken code and add the README-listed features.

• Record a 10-second video after the changes.

• Push updated code to a personal repository.

• Use separate commits for each task; do not raise a pull request to the SETAPESU26 repository.

• Submit before/after videos, updated code, and exported chat history under the Lab-4 folder.

Conversation Work Log

1. Repository and lab requirements

The student shared the assigned repository URL and asked for help finishing the lab. The handout was reviewed and the required workflow and deliverables were outlined.

2. Running the project

Guidance was provided to download/fork the repository, open the extracted project in VS Code, install Pygame if needed, and run the game with `python main.py`. The original game was to be recorded for at least 10 seconds before code edits.

3. Opening an AI assistant in VS Code

The student was guided to open GitHub Copilot Chat, check whether it was installed, and open the full project folder so the assistant could inspect the source files.

4. Task 1 prompt — input deadlock

Prompt used/recommended:

> Inspect game/game_engine.py and fix Task 1 of my Tug of War VibeCoding lab. The alternating A/D keyboard input synchronization can deadlock because of last_key and is_pull_locked. Fix it so rapid alternating inputs register valid pulls without locking the player out. Holding a key should not generate repeated pulls. Preserve the existing Pygame architecture, AI opponent, win detection, and R-to-restart behavior. Explain the root cause, make the actual code changes, and test the behavior. Do not implement the other three tasks yet.

5. Task 2 prompt — dynamic AI panic surges

> Implement Task 2 in game/game_engine.py: dynamic AI panic surges. As the rope marker approaches the player's winning threshold on the left, the computer must react more frequently and pull harder to stage a comeback. Use pygame.time.get_ticks() for timing, keep normal AI behavior when danger is low, and ensure the feature integrates with the existing win detection and reset logic. Do not implement Tasks 3 or 4. Show the code changes and explain how to test the surge.

6. Task 3 prompt — rope tension and leaning animations

> Implement Task 3 in game/rope.py, game/player.py, and game/game_engine.py as needed. Add dynamic rope sag or vibration based on struggle intensity, and animate both puller characters leaning backward based on pulling momentum. Use smooth time-based animations, preserve the existing visual style and gameplay, and avoid blocking the game loop. Explain each file change and provide a test checklist. Do not implement Task 4.

7. Task 4 prompt — match timer and sudden death

> Implement Task 4 of the Tug of War Repair Lab. Add a visible match timer at the top of the game. If neither side wins within 45 seconds, activate a visible SUDDEN DEATH state and double the pulling power of all player and computer pulls. Use elapsed time rather than frame counts. Ensure the timer and sudden death state reset correctly when R is pressed on the Game Over screen, and make sure the game can still end normally. Preserve Tasks 1–3. Explain how to test each requirement.

Troubleshooting and Verification

8. Running from the correct folder

When `python main.py` was run from `C:\Users\Dharshini`, Python reported that the file could not be found. Guidance was given to locate the extracted project folder, change into that directory, and run the command again.

9. Verifying that edits are active

Because the game appeared unchanged, the recommended checks were to save edited files (Ctrl+S), confirm the correct project folder is open, and inspect Git status/diff before assuming the new features work.

10. GitHub permission error and resolution path

The push failed with HTTP 403 because the local origin pointed to the SETAPESU26 repository. The guidance was to commit locally, fork the project to the student's account, update origin to the personal fork, and push there. The local commit and remote push are separate operations.

11. Recommended test checklist

• Keyboard: alternate A and D rapidly; confirm input does not deadlock and holding a key does not create repeated pulls.

• AI surge: push the marker toward the player's winning boundary and observe whether the computer reacts faster and pulls harder.

• Animations: observe rope sag/vibration and both characters' leaning while pulling.

• Timer: allow 45 seconds to pass without a winner; confirm sudden death is displayed and pulling power changes.

• Restart: press R after game over and confirm the timer, marker, and AI state reset.

• Record the after video only after the features have been tested.

Useful terminal commands

```
python main.py
```

```
python -m pip install pygame
```

```
git status
```

```
git diff --stat
```

```
git log --oneline -5
```

```
git remote -v
```

```
git push -u origin main
```

Accuracy note: This log documents the prompts and instructions discussed in this conversation. It does not claim that the code was independently inspected, that all four features passed testing, or that the final video and submission repository were verified. Add screenshots or actual Copilot responses if your instructor requires a complete verbatim AI conversation export.